EXP. NO: 12b
DATE: 15/10/26

IMPLEMENT CHAT CLIENT SERVER USING TCP/UDP SOCKETS

AIM:

To implement a chat client server application using socket programming & enable two way communication between the client and server.

PROCEDURE:

1) Open VS Code
2) Create a Folder
3) Create two Python files.
   - chat_server.py
   - chat_client.py
4) In chat_server.py, create a TCP Socket using:
   - AF_INET
   - SOCK_STREAM
5) Bind the server to a port.
6) Make the server listen for a client.
7) Accept the client connection.
8) In chat_client.py, create a TCP socket.
9) Enter the Server IP address.
10) Connect the client to the server.
11) Start the conversation.
12) The client sends a message to the server.

Explorer

12b

chat_client.py

chat_server.py

chat_client.py

```python
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_ip = input("Enter server IP: ")
client.connect((server_ip, 5001))
while True:
    message = input("Client: ")
    client.send(message.encode())

    if message.lower() == "exit":
        break

    reply = client.recv(1024).decode()
    print("Server:", reply)

    if reply.lower() == "exit":
        break

client.close()
```

Problems Output Debug Console Terminal Ports

```
PS D:\12b> python chat_server.py
Chat server waiting for connection...
Connected to: ('127.0.0.1', 55821)
Client: Hello
Server: Hi
Client: How are you?
Server: Iam Fine!!
```

```
PS D:\12b> python chat_client.py
Enter server IP: 127.0.0.1
Client: Hello
Server: Hi
Client: How are you?
Server: Iam Fine!!
Client:
```

powershell

python

python

Explorer

12b

chat_client.py

chat_server.py

chat_server.py

```python
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("0.0.0.0", 5001))
server.listen(1)
print("Chat server waiting for connection...")
conn, addr = server.accept()
print("Connected to:", addr)
while True:
    message = conn.recv(1024).decode()

    if message.lower() == "exit":
        break
    print("Client:", message)
    reply = input("Server: ")
    conn.send(reply.encode())
    if reply.lower() == "exit":
        break
conn.close()
server.close()
```

Problems Output Debug Console Terminal Ports

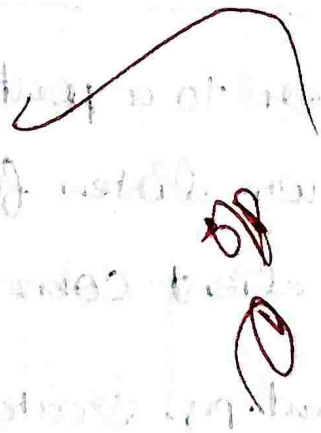
```
PS D:\12b> python chat_server.py
Chat server waiting for connection...
Connected to: ('127.0.0.1', 55821)
Client: Hello
Server: Hi
Client: How are you?
Server: Iam Fine!!
```

```
PS D:\12b> python chat_client.py
Enter server IP: 127.0.0.1
Client: Hello
Server: Hi
Client: How are you?
Server: Iam Fine!!
Client:
```

powershell

python

python



13) The server receives and displays the message.
14) The server enters a reply & sends it to the client.
15) The client receives and displays the server's reply.
16) Continue the process to exchange multiple messages.
17) Enter exit when the conversation is finished.
18) Close both sockets after the chat ends.

RESULT:

Thus, a client server chat application was succesfully implemented using socket programming.